

넘파이 (NumPy) (2)

2026-1학기

ICT융합학부 컴퓨터공학트랙

김대환

목차

1. 배열연산: 브로드캐스팅
2. 비교, 마스크, 부울 로직
3. 팬시 인덱싱
4. 배열 정렬
5. 구조화된 데이터: NumPy의 구조화된 배열

배열연산: 브로드캐스팅

배열연산: 브로드캐스팅 (1)

- 브로드캐스팅 (Broadcasting)

- 서로 다른 크기의 배열을 자동으로 맞춰 연산하는 방법
- 다른 크기의 배열에 이항 유니버설 함수 (덧셈, 뺄셈, 곱셈 등)를 적용하기 위한 규칙의 집합

- 브로드캐스팅 간단 예제

- 같은 크기의 배열에서 이항 연산은 배열의 요소 단위로 수행

```
import numpy as np
```

```
a = np.array([0, 1, 2])  
b = np.array([5, 5, 5])  
a + b
```

```
array([5, 6, 7])
```

값 5를 [5, 5, 5]로 확장하고 복제하고, 그 결과를 더하는 연산으로 생각!!



하지만, 실제로 값 복제가 발생하지는 않음 !!

```
a + 5
```

```
array([5, 6, 7])
```

배열연산: 브로드캐스팅 (2)

■ (예제) 1차원 배열 + 2차원 배열

```
a = np.array([0, 1, 2])
```

```
M = np.ones((3, 3))
```

```
M
```

```
array([[ 1.,  1.,  1.],  
       [ 1.,  1.,  1.],  
       [ 1.,  1.,  1.]])
```

```
M + a
```

```
array([[ 1.,  2.,  3.],  
       [ 1.,  2.,  3.],  
       [ 1.,  2.,  3.]])
```



1차원 배열 a는 M의 형상에 맞추기 위해,
두 번째 차원까지 확장 또는 브로드캐스팅

■ (예제) 두 배열 모두 브로드캐스팅

```
a = np.arange(3)  
b = np.arange(3)[: , np.newaxis]
```

```
print(a)  
print(b)
```

```
[0 1 2]  
[[0]  
 [1]  
 [2]]
```

```
a + b
```

```
array([[0, 1, 2],  
       [1, 2, 3],  
       [2, 3, 4]])
```

```
np.arange(3)+5
```

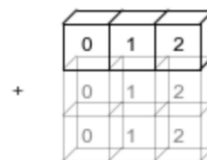
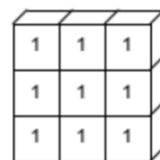


+

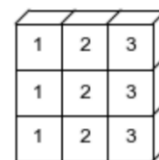


=

```
np.ones((3,3))+np.arange(3)
```

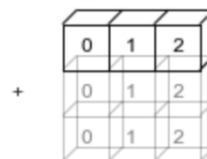
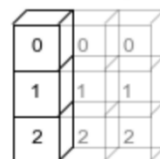


+

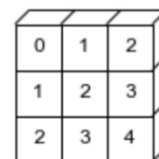


=

```
np.arange(3).reshape((3,1))+np.arange(3)
```



+



=

배열연산: 브로드캐스팅 (3)

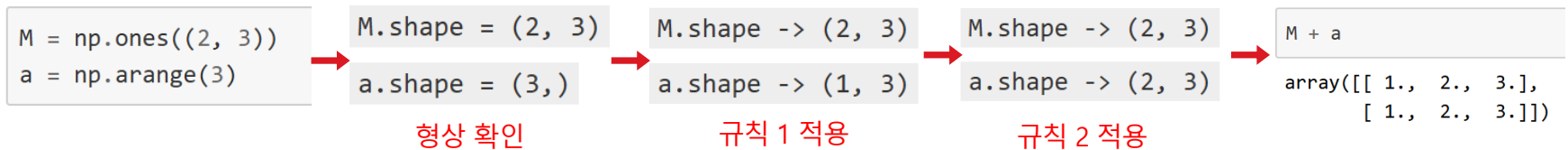
■ 브로드캐스팅 규칙

(규칙1) 두 배열의 차원 수가 다르면 → 더 작은 수의 차원을 가진 배열 형상의 앞쪽(왼쪽)을 1로 채움

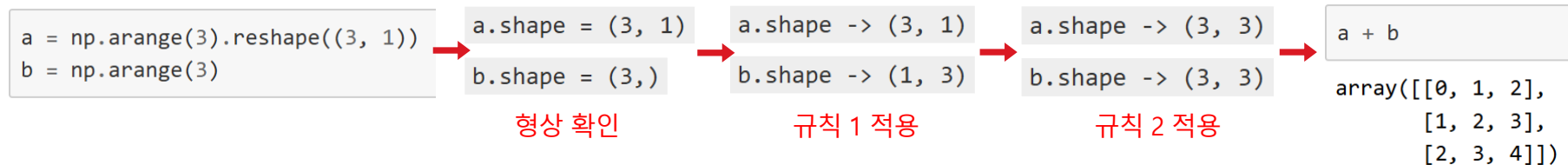
(규칙2) 두 배열의 형상이 어떤 차원에서든 일치하지 않으면 → 해당 차원의 형상이 1인 배열이 다른 형상과 일치하도록 늘어남

(규칙3) 임의의 차원에서 크기가 일치하지 않고 1도 아니라면 → 오류

■ (예제) 1차원 배열 + 2차원 배열



■ (예제) 두 배열 모두 브로드캐스팅



배열연산: 브로드캐스팅 (4)

▪ (예제) 두배열이 호환되지 않는 경우

`M = np.ones((3, 2))`
`a = np.arange(3)` → `M.shape = (3, 2)`
`a.shape = (3,)` → `M.shape -> (3, 2)`
`a.shape -> (1, 3)` → `M.shape -> (3, 2)`
`a.shape -> (3, 3)`

형상 확인 규칙 1 적용

`M + a` 크기 맞지 않음 !! 오류 !!

→ `ValueError` Traceback (most recent call last):
`<ipython-input-13-9e16e9f98da6> in <module>()
----> 1 M + a`

`ValueError: operands could not be broadcast together with shapes (3,2) (3,)`

규칙 3 적용

※ (해결책) `np.newaxis` 사용하여 명시적 형상 변경

`a[:, np.newaxis].shape`

`(3, 1)`

`M + a[:, np.newaxis]`

`array([[1., 1.],
 [2., 2.],
 [3., 3.]])`

배열연산: 브로드캐스팅 (5) – 실전브로드캐스팅

배열을 중앙 정렬하기

```
X = np.random.random((10, 3))
```

`Xmean.shape`

`(1, 3) → (10, 3)`

```
Xmean = X.mean(0)  
Xmean
```

```
X_centered = X - Xmean
```

브로드캐스팅

```
X_centered.mean(0)
```

```
array([ 2.22044605e-17, -7.77156117e-17, -1.66533454e-17])
```

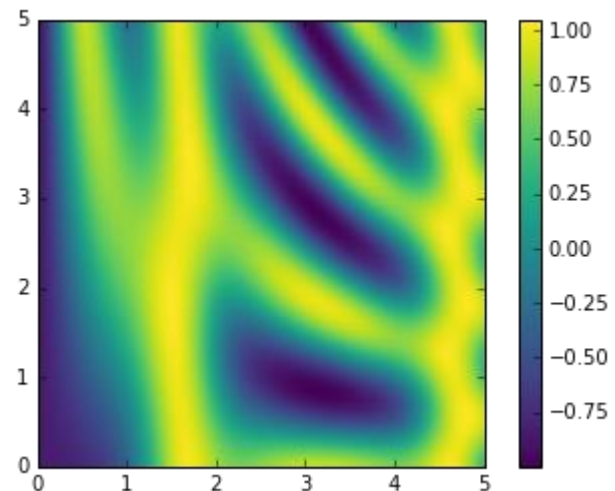
```
array([ 0.53514715,  0.66567217,  0.44385899])
```

2차원 함수 플로팅하기

```
# x and y have 50 steps from 0 to 5  
x = np.linspace(0, 5, 50)  
y = np.linspace(0, 5, 50)[: , np.newaxis]  
  
z = np.sin(x) ** 10 + np.cos(10 + y * x) * np.cos(x)
```

```
%matplotlib inline  
import matplotlib.pyplot as plt
```

```
plt.imshow(z, origin='lower', extent=[0, 5, 0, 5],  
           cmap='viridis')  
plt.colorbar();
```



비교, 마스크, 부울 로직

비교, 마스크, 부울 로직 (1)

■ 부울 마스크

- NumPy 배열 내의 값을 검사하고 조작하는데 부울 마스크 사용법을 배움
- 특정 기준에 따라 배열의 값을 추출하거나 수정, 계산, 조작할 때 사용
 - (예) 특정 값보다 더 큰 값을 모두 세거나, 특정 임계치를 넘어서는 이상치 제거 등

■ (예제) 비 온 날 세기

```
import numpy as np
import pandas as pd

# use pandas to extract rainfall inches as a NumPy array
rainfall = pd.read_csv('data/Seattle2014.csv')['PRCP'].values
inches = rainfall / 254.0 # 1/10mm -> inches
inches.shape
```

2014년 시애틀의 일일 강수량통계치

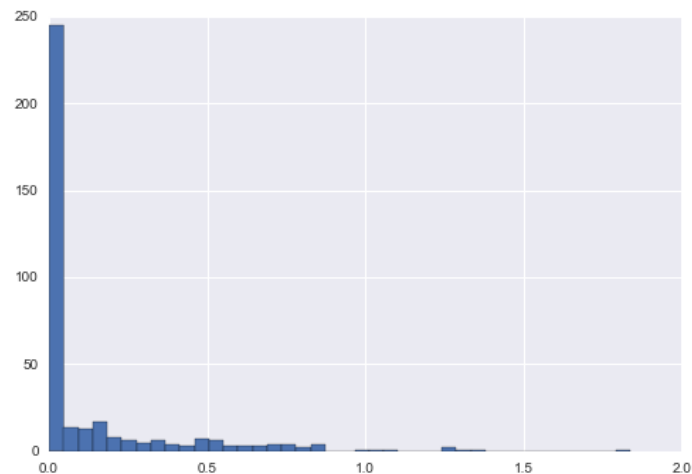
(365,)

```
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn; seaborn.set() # set plot styles
```

```
plt.hist(inches, 40);
```

1. (질문) 그 해 비가 온 날은 며칠일까?
2. (질문) 비 오는 날의 평균 강수량은?
3. (질문) 0.5인치 이상 비가 온 날은 며칠?

How ???



비교, 마스크, 부울 로직 (2)

ufunc 비교연산자

- (복습) 유니버설 함수에서의 산술연산 → 요소 단위로 연산 진행
- ufunc로 비교연산도 구현 가능

```
x = np.array([1, 2, 3, 4, 5])
```

```
x < 3 # less than
```

```
array([ True,  True, False, False, False], dtype=bool)
```

```
x > 3 # greater than
```

```
array([False, False, False,  True,  True], dtype=bool)
```

```
x <= 3 # less than or equal
```

```
array([ True,  True,  True, False, False], dtype=bool)
```

```
x >= 3 # greater than or equal
```

```
array([False, False,  True,  True,  True], dtype=bool)
```

```
x != 3 # not equal
```

```
array([ True,  True, False,  True,  True], dtype=bool)
```

```
x == 3 # equal
```

```
array([False, False,  True, False, False], dtype=bool)
```

```
(2 * x) == (x ** 2)
```

```
array([False,  True, False, False, False], dtype=bool)
```

OperatorEquivalent ufuncOperatorEquivalent ufunc

==	np.equal	!=	np.not_equal
<	np.less	<=	np.less_equal
>	np.greater	>=	np.greater_equal

```
rng = np.random.RandomState(0)
x = rng.randint(10, size=(3, 4))
x
```

```
array([[5, 0, 3, 3],
       [7, 9, 3, 5],
       [2, 4, 7, 6]])
```

```
x < 6
```

```
array([[ True,  True,  True,  True],
       [False, False,  True,  True],
       [ True,  True, False, False]], dtype=bool)
```

비교, 마스크, 부울 로직 (3)

■ 부울 배열로 작업하기 (요소 개수 세기)

- True인 요소 개수를 세는데는 np.count_nonzero (0이 아닌 수를 셈) 유용

```
print(x)
```

```
[[5 0 3 3]
 [7 9 3 5]
 [2 4 7 6]]
```

```
# how many values less than 6?
np.count_nonzero(x < 6)
```

8

방법 1

```
np.sum(x < 6)
```

8

```
# how many values less than 6 in each row?
np.sum(x < 6, axis=1)
```

array([4, 2, 2])

방법 2

- 값 중 하나라도 참이 있는 지나 모든 값이 참인지 확인 → np.any () 또는 np.all () 사용

```
# are there any values greater than 8?
np.any(x > 8)
```

True

```
# are all values less than 10?
np.all(x < 10)
```

True

```
# are all values in each row less than 8?
np.all(x < 8, axis=1)
```

array([True, False, True], dtype=bool)

```
# are there any values less than zero?
np.any(x < 0)
```

False

```
# are all values equal to 6?
np.all(x == 6)
```

False

특정 축을 따라 사용 가능

비교, 마스크, 부울 로직 (4)

■ 부울 배열로 작업하기 (부울 연산자)

- 비트 단위 로직 연산자 &, |, ^, ~ 사용

```
np.sum((inches > 0.5) & (inches < 1))
```

29

```
inches > (0.5 & inches) < 1
```

Error 발생

```
np.sum(~( (inches <= 0.5) | (inches >= 1) ))
```

29

```
print("Number days without rain:      ", np.sum(inches == 0))
print("Number days with rain:         ", np.sum(inches != 0))
print("Days with more than 0.5 inches:", np.sum(inches > 0.5))
print("Rainy days with < 0.2 inches  :", np.sum((inches > 0) &
                                                (inches < 0.2)))
```

```
Number days without rain:      215
Number days with rain:         150
Days with more than 0.5 inches: 37
Rainy days with < 0.2 inches  : 75
```

OperatorEquivalent ufuncOperatorEquivalent ufunc

&	np.bitwise_and		np.bitwise_or
^	np.bitwise_xor	~	np.bitwise_not

비교, 마스크, 부울 로직 (5)

■ 마스크로서의 부울 배열

- 부울 배열을 마스크로 사용해 데이터 자체의 특정 부분 집합을 선택

x

```
array([[5, 0, 3, 3],  
       [7, 9, 3, 5],  
       [2, 4, 7, 6]])
```

x < 5

```
array([[False,  True,  True,  True],  
       [False, False,  True, False],  
       [ True,  True, False, False]], dtype=bool)
```

x[x < 5]

```
array([0, 3, 3, 3, 2, 4])
```

마스크 배열의 True 위치
데이터 값들로 채워짐

```
# construct a mask of all rainy days  
rainy = (inches > 0)  
  
# construct a mask of all summer days (June 21st is the 172nd day)  
days = np.arange(365)  
summer = (days > 172) & (days < 262)  
  
print("Median precip on rainy days in 2014 (inches):  ",  
      np.median(inches[rainy]))  
print("Median precip on summer days in 2014 (inches): ",  
      np.median(inches[summer]))  
print("Maximum precip on summer days in 2014 (inches): ",  
      np.max(inches[summer]))  
print("Median precip on non-summer rainy days (inches):",  
      np.median(inches[rainy & ~summer]))
```

```
Median precip on rainy days in 2014 (inches):    0.194881889764  
Median precip on summer days in 2014 (inches):    0.0  
Maximum precip on summer days in 2014 (inches):  0.850393700787  
Median precip on non-summer rainy days (inches): 0.200787401575
```

비교, 마스크, 부울 로직 (6)

- 키워드 `and/or` vs 연산자 `&/|` 사용
 - `and/or` : 전체 객체의 참과 거짓 측정
 - `&/|` : 각 객체의 비트를 참조

```
bool(42), bool(0)
```

```
(True, False)
```

```
bool(42 and 0)
```

```
False
```

```
bool(42 or 0)
```

```
True
```

VS

```
bin(42)
```

```
'0b101010'
```

```
bin(59)
```

```
'0b111011'
```

```
bin(42 & 59)
```

```
'0b101010'
```

```
bin(42 | 59)
```

```
'0b111011'
```

비교, 마스크, 부울 로직 (7)

■ 키워드 and/or vs 연산자 &| 사용 (2)

```
A = np.array([1, 0, 1, 0, 1, 0], dtype=bool)
B = np.array([1, 1, 1, 0, 1, 1], dtype=bool)
A | B
```

```
array([ True,  True,  True, False,  True,  True], dtype=bool)
```

← 대응하는 비트들끼리 연산 진행

```
A or B
```

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-38-5d8e4f2e21c0> in <module>()
----> 1 A or B
```

```
ValueError: The truth value of an array with more than one element is ambiguous
◀────────────────────────────────────────────────────────────────────────────────▶
```

← or는 전체 배열의 참/거짓을 판단하려고 하니, Error 발생

```
x = np.arange(10)
(x > 4) & (x < 8)
```

```
array([False, False, False, False, False,  True,  True,  True, False, False])
◀────────────────────────────────────────────────────────────────────────────────▶
```

← 특정 배열 부울식 &| 사용

```
(x > 4) and (x < 8)
```

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-40-3d24f1ffd63d> in <module>()
----> 1 (x > 4) and (x < 8)
```

```
ValueError: The truth value of an array with more than one element is ambiguous
◀────────────────────────────────────────────────────────────────────────────────▶
```

← Error 발생. 이유는??

팬시 인덱싱

팬시 인덱싱 (1)

■ 팬시 인덱싱 (Fancy Indexing)

- 단순 인덱싱과 비슷하지만, 단일 스칼라 대신 인덱스 배열을 전달
- 즉, 한 번에 여러 배열 요소에 접근하기 위함
- 복잡한 배열 값의 하위 집합에 매우 빠르게 접근 가능

```
import numpy as np
rand = np.random.RandomState(42)

x = rand.randint(100, size=10)
print(x)
```

[51 92 14 71 60 20 82 86 74 74]

```
[x[3], x[7], x[2]]
```

[71, 86, 14]

```
ind = [3, 7, 4]
x[ind]
```

array([71, 86, 60])

```
ind = np.array([[3, 7],
                [4, 5]])
x[ind]
```

array([[71, 86],
 [60, 20]])

← 인덱싱 배열의 형상 반영

팬시 인덱싱 (2)

■ 팬시 인덱싱 (Fancy Indexing) (2)

```
X = np.arange(12).reshape((3, 4))  
X
```

```
array([[ 0,  1,  2,  3],  
       [ 4,  5,  6,  7],  
       [ 8,  9, 10, 11]])
```

```
row = np.array([0, 1, 2])  
col = np.array([2, 1, 3])  
X[row, col]
```

```
array([ 2,  5, 11])
```

← 여러 차원에서도 사용 가능
X[0, 2], X[1, 1], X[2, 3]

```
X[row[:, np.newaxis], col]
```

```
array([[ 2,  1,  3],  
       [ 6,  5,  7],  
       [10,  9, 11]])
```

```
row = [[ 0],  
       [ 1],  
       [ 2]]
```

+

```
col = [ 2, 1, 3]
```

← 브로드캐스팅을 사용

```
row[:, np.newaxis] * col
```

```
array([[0, 0, 0],  
       [2, 1, 3],  
       [4, 2, 6]])
```

```
row = [[ 0],  
       [ 1],  
       [ 2]]
```

*

```
col = [ 2, 1, 3]
```

팬시 인덱싱 (3)

■ 결합 인덱싱

- 팬시 인덱스와 단순 인덱스 결합 가능

```
print(X)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

```
X[2, [2, 0, 1]]
```

```
array([10,  8,  9])
```

```
X[1:, [2, 0, 1]]
```

```
array([[ 6,  4,  5],
       [10,  8,  9]])
```

```
mask = np.array([1, 0, 1, 0], dtype=bool)
X[row[:, np.newaxis], mask]
```

```
array([[ 0,  2],
       [ 4,  6],
       [ 8, 10]])
```

■ (예제) 임의의 점 선택하기

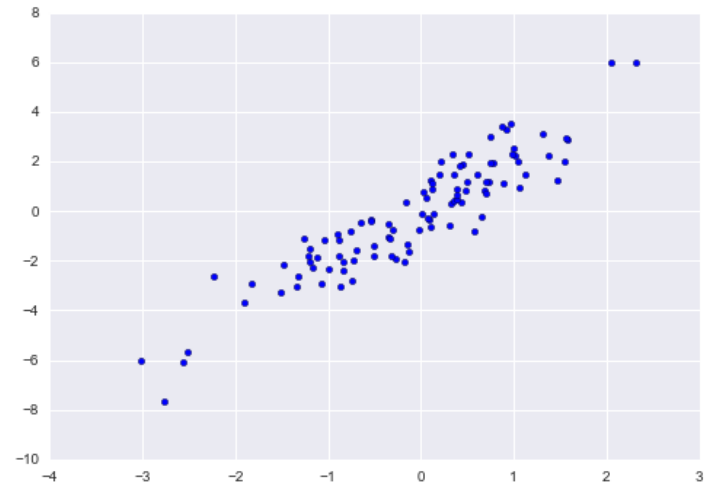
- 2차원 정규분포 → N x D 행렬

```
mean = [0, 0]
cov = [[1, 2],
       [2, 5]]
X = rand.multivariate_normal(mean, cov, 100)
X.shape

(100, 2)
```

```
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn; seaborn.set() # for plot styling

plt.scatter(X[:, 0], X[:, 1]);
```



팬시 인덱싱 (4)

- (예제) 임의의 점 선택하기 (cont')

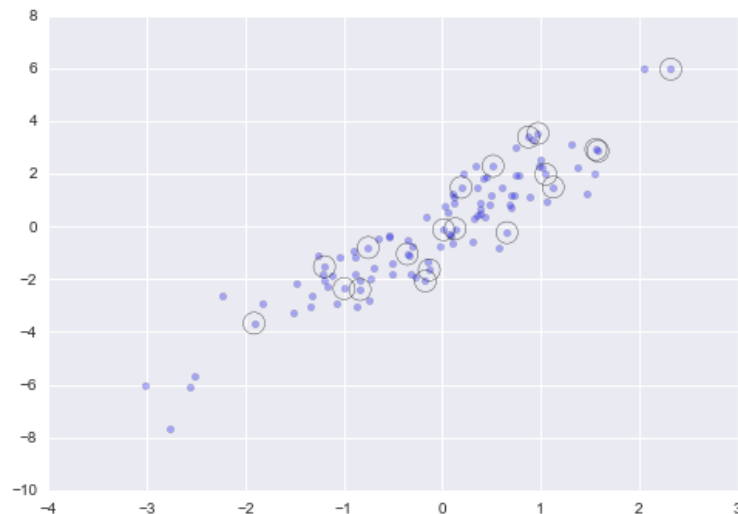
```
indices = np.random.choice(X.shape[0], 20, replace=False)
indices
```

```
array([93, 45, 73, 81, 50, 10, 98, 94,  4, 64, 65, 89, 47, 84, 82, 80, 25,
       90, 63, 20])
```

```
selection = X[indices] # fancy indexing here
selection.shape
```

```
(20, 2)
```

```
plt.scatter(X[:, 0], X[:, 1], alpha=0.3)
plt.scatter(selection[:, 0], selection[:, 1],
            facecolor='none', s=200);
```



팬시 인덱싱 (5)

- 팬시 인덱싱으로 값 변경하기
 - 배열의 일부를 수정하는데도 사용 가능

```
x = np.arange(10)
i = np.array([2, 1, 8, 4])
x[i] = 99
print(x)
```

```
[ 0 99 99  3 99  5  6  7 99  9]
```

```
x[i] -= 10
print(x)
```

```
[ 0 89 89  3 89  5  6  7 89  9]
```

```
x = np.zeros(10)
x[[0, 0]] = [4, 6]
print(x)
```

```
[ 6.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
```

x[0]=4 이후 x[0] = 6 실행

```
i = [2, 3, 3, 4, 4, 4]
x[i] += 1
x
```

```
array([ 6.,  0.,  1.,  1.,  1.,  0.,  0.,  0.,  0.,  0.])
```

← x[3] += 1 이 두 번 실행 되었는데 ? Why ??

```
x = np.zeros(10)
np.add.at(x, i, 1)
print(x)
```

```
[ 0.  0.  1.  2.  3.  0.  0.  0.  0.  0.]
```

← at() 메소드: 지정한 값을 가진 특정 인덱스에
해당 연산자를 즉시 적용

팬시 인덱싱 (6)

▪ (예제) 데이터 구간화

- 데이터를 효율적으로 구간화(binning) 해서 직접 히스토그램 생성

```
np.random.seed(42)
x = np.random.randn(100)

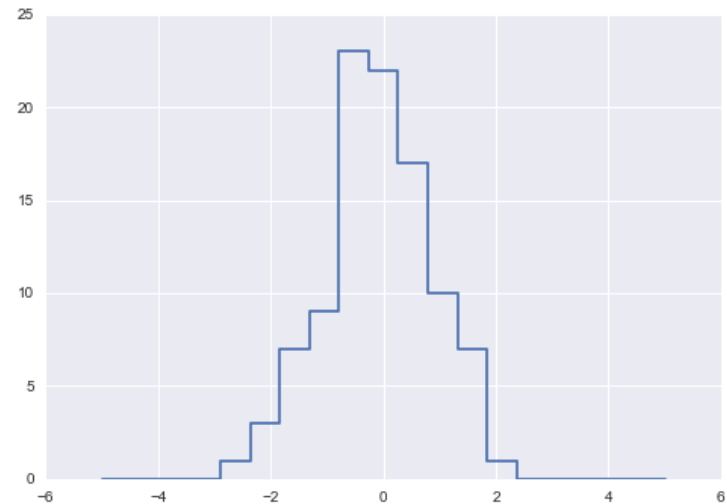
# compute a histogram by hand
bins = np.linspace(-5, 5, 20)
counts = np.zeros_like(bins)

# find the appropriate bin for each x
i = np.searchsorted(bins, x)

# add 1 to each of these bins
np.add.at(counts, i, 1)
```

```
# plot the results
plt.plot(bins, counts, linestyle='steps');
```

or `plt.hist(x, bins, histtype='step');`



팬시 인덱싱 (6)

- (예제) 데이터 구간화 (cont')
 - np.histogram과 비교

```
print("NumPy routine:")
%timeit counts, edges = np.histogram(x, bins)

print("Custom routine:")
%timeit np.add.at(counts, np.searchsorted(bins, x), 1)
```

NumPy routine:
10000 loops, best of 3: 97.6 μ s per loop
Custom routine:
10000 loops, best of 3: 19.5 μ s per loop

VS

개수가 많아질 때 성능이 좋아지도록 설계

```
x = np.random.randn(1000000)
print("NumPy routine:")
%timeit counts, edges = np.histogram(x, bins)

print("Custom routine:")
%timeit np.add.at(counts, np.searchsorted(bins, x), 1)
```

NumPy routine:
10 loops, best of 3: 68.7 ms per loop
Custom routine:
10 loops, best of 3: 135 ms per loop

배열 정렬

배열 정렬 (1)

- 배열 정렬 (Sorting Arrays)

- NumPy 배열의 값을 정렬하는 알고리즘
- 삽입 정렬 (Insertion sorts), 선택 정렬 (Selection sorts), 병합 정렬 (Merge sorts), 퀵 정렬 (Quick sorts), 버블 정렬 (Bubble sorts) 등

- 파이썬 지원 정렬 함수들

- 리스트와 기타 이터러블 객체를 정렬하는 기능의 기본 제공 함수/메소드 존재

```
L = [3, 1, 4, 1, 5, 9, 2, 6]  
sorted(L)
```

```
L.sort()  
print(L)
```

[1, 1, 2, 3, 4, 5, 6, 9]

```
sorted('python')
```

['h', 'n', 'o', 'p', 't', 'y']

배열 정렬 (2)

- NumPy의 빠른 정렬

- `np.sort()` : 파이썬 `sorted()`와 유사, 배열의 정렬된 복사본을 반환

```
x = np.array([2, 1, 4, 3, 5])  
np.sort(x)
```

```
array([1, 2, 3, 4, 5])
```

```
x.sort()  
print(x)
```

```
[1 2 3 4 5]
```

```
x = np.array([2, 1, 4, 3, 5])  
i = np.argsort(x)  
print(i)
```

```
[1 0 3 2 4]
```

`argsort()` : 정렬된 요소의 인덱스를 반환

```
x[i]
```

```
array([1, 2, 3, 4, 5])
```

배열 정렬 (3)

- 행이나 열 기준으로 정렬하기

- axis 인수를 사용해 다차원 배열의 특정 행이나 열에 따라 정렬 가능

```
rand = np.random.RandomState(42)
X = rand.randint(0, 10, (4, 6))
print(X)
```

```
[[6 3 7 4 6 9]
 [2 6 7 4 3 7]
 [7 2 5 4 1 7]
 [5 1 4 0 9 5]]
```

```
# sort each column of X
np.sort(X, axis=0)
```

```
array([[2, 1, 4, 0, 1, 5],
       [5, 2, 5, 4, 3, 7],
       [6, 3, 7, 4, 6, 7],
       [7, 6, 7, 4, 9, 9]])
```

```
# sort each row of X
np.sort(X, axis=1)
```

```
array([[3, 4, 6, 6, 7, 9],
       [2, 3, 4, 6, 7, 7],
       [1, 2, 4, 5, 7, 7],
       [0, 1, 4, 5, 5, 9]])
```

배열 정렬 (3)

■ 부분 정렬: 파티션 나누기

- 단순히 배열에서 K개의 가장 작은 값을 찾을 때 사용
- `np.partition()`: 배열과 숫자 K를 취해 새로운 배열 반환, 반환된 파티션의 왼쪽에는 K개의 가장 작은 값이 있고, 오른쪽에는 나머지 값이 임의의 순서로 나열

```
x = np.array([7, 2, 3, 1, 6, 5, 4])  
np.partition(x, 3)
```

```
array([2, 1, 3, 4, 6, 5, 7])
```

```
np.partition(X, 2, axis=1)
```

```
array([[3, 4, 6, 7, 6, 9],  
       [2, 3, 4, 7, 6, 7],  
       [1, 2, 4, 5, 7, 7],  
       [0, 1, 4, 5, 9, 5]])
```

← 다차원 배열의 임의의 축을 따라 파티션 가능

← 행에 대해서 `partition()` 적용

배열 정렬 (5)

■ (예제) k-최근접 이웃 알고리즘 (cont')

```
dist_sq = np.sum((X[:, np.newaxis, :] - X[np.newaxis, :, :]) ** 2, axis=-1)
```

단계별로 나누어 계산 →

```
# for each pair of points, compute differences in their coordinates  
differences = X[:, np.newaxis, :] - X[np.newaxis, :, :]  
differences.shape
```

```
(10, 10, 2)
```

```
# square the coordinate differences  
sq_differences = differences ** 2  
sq_differences.shape
```

```
(10, 10, 2)
```

```
# sum the coordinate differences to get the squared distance  
dist_sq = sq_differences.sum(-1)  
dist_sq.shape
```

```
(10, 10)
```

배열 정렬 (6)

■ (예제) k-최근접 이웃 알고리즘 (cont')

```
dist_sq.diagonal()

array([ 0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.])
```

```
nearest = np.argsort(dist_sq, axis=1)
print(nearest)
```

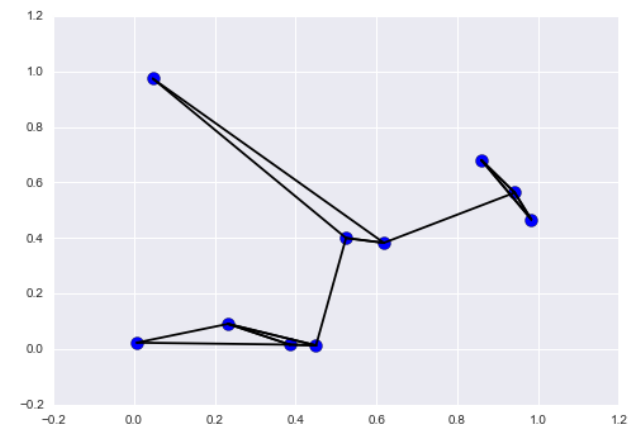
```
[[0 3 9 7 1 4 2 5 6 8]
 [1 4 7 9 3 6 8 5 0 2]
 [2 1 4 6 3 0 8 9 7 5]
 [3 9 7 0 1 4 5 8 6 2]
 [4 1 8 5 6 7 9 3 0 2]
 [5 8 6 4 1 7 9 3 2 0]
 [6 8 5 4 1 7 9 3 2 0]
 [7 9 3 1 4 0 5 8 6 2]
 [8 5 6 4 1 7 9 3 2 0]
 [9 7 3 0 1 4 5 8 6 2]]
```

```
K = 2
nearest_partition = np.argpartition(dist_sq, K + 1, axis=1)
```

```
plt.scatter(X[:, 0], X[:, 1], s=100)

# draw lines from each point to its two nearest neighbors
K = 2

for i in range(X.shape[0]):
    for j in nearest_partition[i, :K+1]:
        # plot a line from X[i] to X[j]
        # use some zip magic to make it happen:
        plt.plot(*zip(X[j], X[i]), color='black')
```



구조화된 데이터: NumPy의 구조화된 배열

구조화된 배열: NumPy의 구조화된 배열 (1)

■ 구조화된 배열

- NumPy에서 복합적인 이종 데이터를 효율적으로 저장하기 위한 구조화된 배열과 레코드 배열에 대해 설명

```
import numpy as np
```

```
name = ['Alice', 'Bob', 'Cathy', 'Doug']  
age = [25, 45, 37, 19]  
weight = [55.0, 85.5, 68.0, 61.5]
```

```
x = np.zeros(4, dtype=int)
```

```
# Use a compound data type for structured arrays  
data = np.zeros(4, dtype={'names':('name', 'age', 'weight'),  
                          'formats':('U10', 'i4', 'f8')})  
print(data.dtype)
```

```
[('name', '<U10'), ('age', '<i4'), ('weight', '<f8')]
```

```
data['name'] = name  
data['age'] = age  
data['weight'] = weight  
print(data)
```

```
[('Alice', 25, 55.0) ('Bob', 45, 85.5) ('Cathy', 37, 68.0)  
 ('Doug', 19, 61.5)]
```

- 'U10': 최대 길이 10을 갖는 유니코드 문자열
- 'i4': 4바이트 (32비트) 정수
- 'f8': 8바이트 (64비트) 부동 소수점

구조화된 배열: NumPy의 구조화된 배열 (2)

■ 구조화된 배열 만들기

```
# Get all names
data['name']
```

```
array(['Alice', 'Bob', 'Cathy', 'Doug'],
      dtype='<U10')
```

```
# Get first row of data
data[0]
```

```
('Alice', 25, 55.0)
```

```
# Get the name from the last row
data[-1]['name']
```

```
'Doug'
```

```
# Get names where age is under 30
data[data['age'] < 30]['name']
```

```
array(['Alice', 'Doug'],
      dtype='<U10')
```

Character	Description	Example
'b'	Byte	<code>np.dtype('b')</code>
'i'	Signed integer	<code>np.dtype('i4') == np.int32</code>
'u'	Unsigned integer	<code>np.dtype('u1') == np.uint8</code>
'f'	Floating point	<code>np.dtype('f8') == np.float64</code>
'c'	Complex floating point	<code>np.dtype('c16') == np.complex128</code>
'S', 'a'	String	<code>np.dtype('S5')</code>
'U'	Unicode string	<code>np.dtype('U') == np.str_</code>
'V'	Raw data (void)	<code>np.dtype('V') == np.void</code>

```
np.dtype({'names':('name', 'age', 'weight'),
         'formats':('U10', 'i4', 'f8')})
```

```
dtype([('name', '<U10'), ('age', '<i4'), ('weight', '<f8')])
```

```
np.dtype({'names':('name', 'age', 'weight'),
         'formats':((np.str_, 10), int, np.float32)})
```

```
dtype([('name', '<U10'), ('age', '<i8'), ('weight', '<f4')])
```

→ 숫자 타입은 파이썬 타입이나 NumPy dtype으로 지정

```
np.dtype([('name', 'S10'), ('age', 'i4'), ('weight', 'f8')])
```

```
dtype([('name', 'S10'), ('age', '<i4'), ('weight', '<f8')])
```

```
np.dtype('S10,i4,f8')
```

```
dtype([('f0', 'S10'), ('f1', '<i4'), ('f2', '<f8')])
```

→ 복합 타입은 튜플의 리스트로 지정

- '<' 과 '>'은 리틀 엔디안과 빅엔디안
- 리틀엔디안은 작은 값(하위 바이트)이 먼저 저장
- 빅엔디안은 큰 값(상위 바이트)이 먼저 저장

구조화된 배열: NumPy의 구조화된 배열 (3)

■ 고급 복합 타입

- 더 복잡한 복합 타입을 정의 가능
- (예) 각 요소가 값의 배열이나 행렬을 담고 있는 타입

```
tp = np.dtype([('id', 'i8'), ('mat', 'f8', (3, 3))])  
X = np.zeros(1, dtype=tp)  
print(X[0])  
print(X['mat'][0])
```

3 x 3의 부동 소수점 행렬

```
(0, [[0.0, 0.0, 0.0], [0.0, 0.0, 0.0], [0.0, 0.0, 0.0]])  
[[ 0.  0.  0.]  
 [ 0.  0.  0.]  
 [ 0.  0.  0.]]
```

■ 레코드 배열: 트위스트를 가진 구조화된 배열

- 딕셔너리 키 대신 속성으로 필드에 접근할 수 있는 np.recarray 클래스 제공

```
data['age']
```

```
array([25, 45, 37, 19], dtype=int32)
```

```
data_rec = data.view(np.recarray)  
data_rec.age
```

```
array([25, 45, 37, 19], dtype=int32)
```

```
%timeit data['age']  
%timeit data_rec['age']  
%timeit data_rec.age
```

```
1000000 loops, best of 3: 241 ns per loop  
100000 loops, best of 3: 4.61 µs per loop  
100000 loops, best of 3: 7.27 µs per loop
```

Q & A
